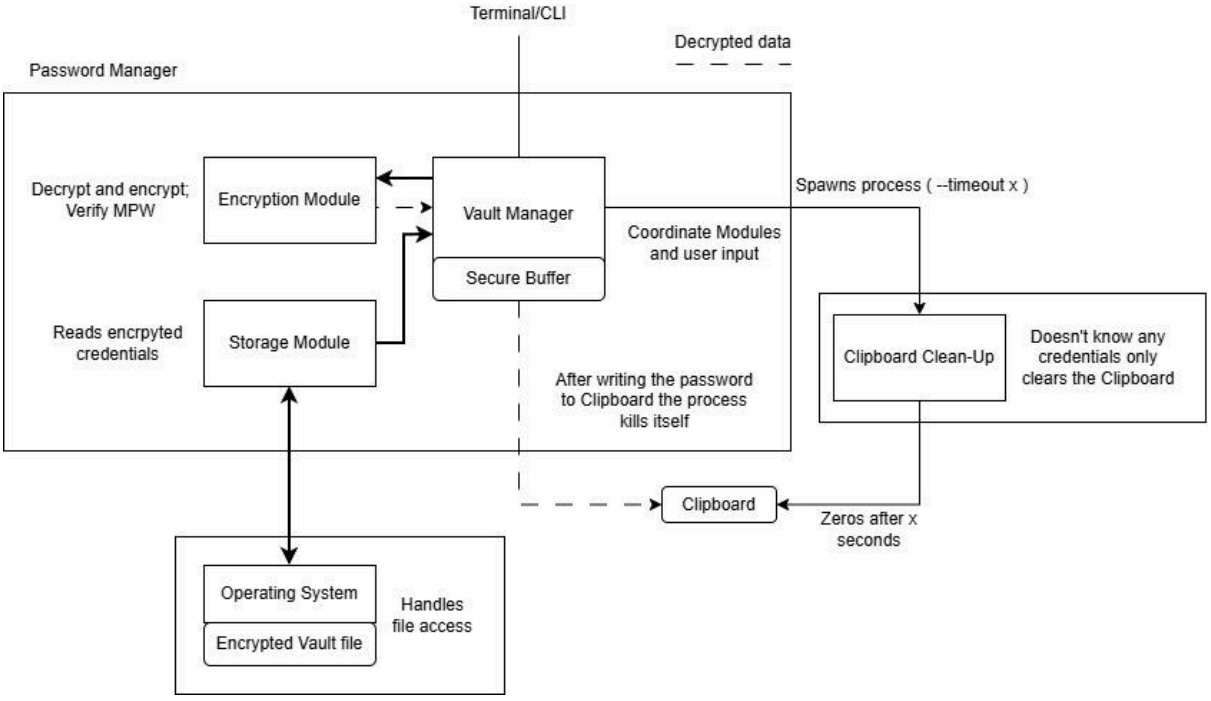
Password Manager

This project includes the designing of a local offline password manager. The focus of this project lies on developing a software with security in mind, from the beginning, protecting software from unauthorized access and modifications and ensuring data integrity, confidentiality and availability.

This project was developed during the autumn semester of 2026 as part of the “Secure Programming” course by Julian Lorenz, an Erasmus exchange student from Germany.

Software Architecture



Components

Components the System will use
CLI / User Interface: Parses commands, requests user input and displays operation results.

Vault Manager: Coordinates the application workflow, including vault access, credential operations and error handling.

Cryptographer: Derives the vault key from the master password using Argon2id and performs authenticated encryption and decryption.

Storage: Reads and writes the vault file and retrieves credential records by account identifier. It does not decrypt credentials.

Secure Memory: Provides controlled memory buffers for sensitive values, helping minimize plaintext copies and allowing sensitive data to be zeroized after use.

Clipboard and Cleanup Process: Copies requested credentials to the operating system clipboard. A separate cleanup process attempts to clear the clipboard after 15 seconds if its contents have not changed.

Data Flow

Retrieving a Credential

For a command such as `pwm --get-pw --account "youtube"`, the application follows this sequence:

1. The CLI parses the command and requests the master password through hidden input.
2. The Vault Manager obtains the vault metadata from Storage and passes the master password, salt and KDF parameters to the Cryptographer.
3. The Cryptographer derives the vault key using Argon2id and authenticates and decrypts the vault check.
4. If validation fails, the operation is aborted. Otherwise, the Vault Manager asks Storage to retrieve the encrypted record matching the requested account identifier.
5. The Cryptographer decrypts the credential directly into a controlled sensitive memory buffer.
6. The credential is copied to the clipboard by default. If the user explicitly requests terminal display, the CLI displays it instead.
7. If the clipboard was used, the Vault Manager starts the cleanup process.
8. Sensitive buffers are zeroized, and the main PWM process terminates.

If the requested account does not exist, Storage reports an account-not-found error and the operation terminates without returning a credential.

Creating a Credential

When creating a new credential, the PWM requests the master password, derives the vault key and validates the vault check before proceeding. The password generator creates the new credential, which is encrypted by the Cryptographer and stored by Storage as a new record. The generated credential is then provided to the user according to the selected output behavior.

Initializing a Vault

During initial setup, the user enters the master password twice. The application compares the inputs and aborts initialization if they do not match. If they match, the application generates a random salt, establishes the Argon2id parameters, derives the vault key and creates the authenticated encrypted vault check. The vault is then initialized with its metadata and an initially empty set of credential entries.

Threat Model

Component	Threat	Mitigation
Master Password	Brute-force / offline guessing	Argon2id + unique salt + appropriate parameters
Master Password	Exposure via CLI/processes	Never pass password as CLI argument; hidden input
Vault at rest	Stolen vault file	Authenticated encryption
Vault at rest	Vault tampering	AEAD authentication
Vault at rest	Weak file permissions	Restrictive filesystem permissions
Vault in memory	Plaintext credential exposure	Secure memory, minimize lifetime, zeroization
Vault in memory	Key exposure	Vault key only exists during operation, then zeroized
Interface	Password visible in terminal	Copy to clipboard by default; explicit `--show-pw`
Interface	Clipboard exposure	Clear after 15 s
Interface	Malicious/malformed input	Strict validation / safe parsing

Design Decisions

Decision	Rationale
Single-user vault	Avoids unnecessary account-management complexity.
One operation per process	Minimizes the lifetime of sensitive data in memory and eliminates persistent sessions.
Argon2id for key derivation	Makes offline master-password guessing more expensive.
Single vault key	Simplifies key management by avoiding an additional key hierarchy.
Vault check	Verifies the master password before accessing or modifying credentials.
Per-entry authenticated encryption	Protects credential confidentiality and integrity while allowing individual entry lookups.
Plaintext account identifiers	Enables simple lookups without decrypting the vault; account names are intentionally exposed.
Clipboard by default	Reduces accidental password exposure in terminal output. A cleanup process attempts to clear the clipboard after 15 seconds.
Established cryptographic library	Avoids the risks of implementing cryptographic primitives manually.
Controlled sensitive memory	Minimizes plaintext copies and lifetime, and zeroizes sensitive buffers after use.